Secure Cross-Silo Synthetic Genomic Data Generation

Daniil Filienko, Martine De Cock, Sikha Pentyala
University of Washington Tacoma
Corresponding author: Daniil Filienko, daniilf@uw.edu

Abstract

Access to genomic data is highly regulated due to its sensitive nature. While safeguards are essential, cumbersome data access processes pose a significant barrier to the development of AI methods for genomics. Synthetic data generation can mitigate this tension by enabling broader data sharing without exposing sensitive information. Synthetic genomic data are produced by training generative models on real data and subsequently sampling artificial data that preserves relevant statistics while limiting disclosures about the underlying individuals. In some settings, a single data holder may have sufficient data to train such generative models; however, in many applications data must be combined across multiple sites to achieve adequate scale. This need arises, e.g., in rare disease studies, where individual hospitals typically hold data for only a small number of patients. The solution we present in this paper enables multiple data holders to jointly train a synthetic data generator without revealing their raw data. Our approach combines secure multiparty computation (MPC) to ensure input privacy, so that no party ever discloses its data in unencrypted form, with differential privacy (DP) to provide output privacy by mitigating information leakage from the released synthetic data. We empirically demonstrate the effectiveness of the proposed method by generating high-utility synthetic datasets from multiple real RNA-seq cohorts in federated settings, showing that our approach enables privacy-preserving data synthesis even when data are distributed across institutions.

1 Introduction

Patient data, and genomic data in particular, is subject to strict access controls. Even when datasets are advertised as "open", researchers are often required to navigate long and cumbersome access procedures, such as negotiating complex contractual agreements with data controllers, completing extensive training requirements, or performing analyses within controlled data enclaves [28]. In contrast, AI researchers are used to minimal barriers when accessing, exploring, and experimenting with new datasets. This mismatch significantly constrains AI work in biomedicine. At the same time, datasets containing protected health information cannot be made publicly available without careful safeguards. Concerns regarding the privacy and confidentiality of clinical and genomic data are well justified and have led to increasingly strict controlled-access requirements [19, 20, 21].

A promising method for operationalizing sensitive patient data for research while protecting patient privacy is synthetic data generation (SDG). A common approach to SDG is to train a generative model on real data and subsequently use the trained model to create artificial data that preserves important characteristics of the real data without disclosing sensitive information about individuals [10]. While in some scenarios a single data holder (e.g., a hospital) may have sufficient data on its own to enable SDG, in many cases data from multiple sites needs to be combined. Such federated settings are common, for instance, with rare diseases, where each clinical site has data for a small patient cohort.

In this paper we present a method for training privacy-preserving synthetic RNA sequencing (RNA-seq) data generators in federated settings. Our approach adopts cryptographic techniques based on secure multiparty computation (MPC) [5] to support a federated training process in which each data holder encrypts their data on site before sending it as input. MPC servers perform training operations on the input data while it stays encrypted, thus providing *input privacy.* Furthermore, we use differential privacy (DP) [7] to provide formal, individual-level privacy guarantees for the released synthetic data, thus providing *output privacy.*

The heart of our solution is an MPC-protocol for Private-PGM. Private-PGM is a marginal-based SDG algorithm that learns a probabilisitic graphical model (PGM) from marginal probability distributions estimated on (a discretized version of) the real data, and subsequently samples from the model to generate synthetic data [16]. Private-PGM offers formal DP guarantees by perturbing the estimated marginals with noise. It has been shown to work well for generation of synthetic bulk RNA-seq data in *centralized* settings, i.e., where all the real data resides with one data holder [4, 18]. Our extension of Private-PGM to federated settings consists of cryptographic protocols for quantile binning and estimating marginal probability distributions over the combined data of the data custiodians while it remains encrypted. To this end, we draw inspiration from MPC protocols in the `CaPS` framework previously proposed by Pentyala et al. [23] for the marginal-based SDG algorithms

MWEM-PGM and AIM [16, 17]. We found, however, that a straightforward implementation of these MPC protocols for Private-PGM does not scale well with the high dimensionality of RNA-seq data. Different from Fu et al., who recently proposed to address the inefficiencies of `CaPS` by optimizing secure sort-and-count operations [9], we replace iterative "per sample per gene" computations by one dot product over a vector of all samples per gene. Using a variety of TCGA and leukemia datasets, we demonstrate that our approach for federated synthetic RNA-seq data generation can produce synthetic data in as little as a few minutes, depending on the MPC scheme, with a level of quality that is at par with the centralized setting.

2 Preliminaries

2.1 Secure Multiparty Computation for Input Privacy

Secure multiparty computation (MPC) is an umbrella term for cryptographic approaches that allow two or more parties to jointly compute a specified output from distributed private information without revealing it to one another [5]. In this paper, we follow the "MPC as a service" paradigm, where data holders delegate the computations to a set of non-colluding, but otherwise non-trusted servers. We adopt secret-sharing based MPC where each data holder encrypts their private data x by splitting it into shares (see below for an example) and distributing the shares among a set of MPC servers $\mathcal{S} = \{S_1, S_2, \ldots, S_K\}$. Though x can be reconstructed when all shares are combined, none of the MPC servers by themselves learn anything about the value of x. These servers run MPC protocols to jointly perform computations over the secret shares, in our case computations for generating synthetic data. As all computations are done over the secret shares, the servers do not learn the values of the inputs nor of intermediate results, i.e., MPC provides *input privacy*.

The security guarantees of an MPC protocol are defined by the type of adversary it can withstand. These are typically categorized into passive and active threat models, reflecting the power an adversary has over the MPC servers it corrupts. In the *passive* (or "semi-honest") setting, all parties follow the instructions of the protocol, but the adversary attempts to learn private information from the internal state of the corrupted servers and the messages that they receive. MPC protocols that are secure against passive adversaries prevent such leakage of information. In contrast, an *active* (or "malicious") adversary may corrupt an MPC server to arbitrarily deviate from the protocol, including colluding with others to breach privacy or corrupt intermediate computations. One way to detect and prevent this is by using a "brain-and-pinky" architecture [8]. For each virtual communication, one real party referred to as "brain" is elected to transmit all the messages to all real parties in the receiving virtual party, referred to as the "pinky". Right before the final outputs are revealed, all the parties perform a single consistency check. If a malicious "brain" deviated from the protocol and sent a corrupted message, it will conflict with at least one of the "pinky's" locally computed views. Such inconsistency would trigger an immediate abort before any private data is leaked or an incorrect result is accepted.

The protocols that we propose in Section 4 are sufficiently generic to be used in settings with passive or active adversaries. This is achieved by changing the underlying MPC scheme to align with the desired security setting. To give the reader a concrete understanding, we give a brief description of a specific MPC scheme based on replicated secret sharing with $K = 3$ computing parties [1]. MPC works for inputs defined over a finite field or ring. Since inputs for SDG algorithms are finite precision real numbers, the data holders convert all of their inputs to fixed precision [3] and map these to integers modulo q, i.e., to the ring $\mathbb{Z}_q = \{0, 1, \ldots, q-1\}$, with q a power of 2. A private value x in $\mathbb{Z}_q$ is secret shared among servers (parties) S_1, S_2, and S_3 by selecting uniformly random shares $x_1, x_2, x_3 \in \mathbb{Z}_q$ such that $x_1 + x_2 + x_3 = x \mod q$, and giving (x_1, x_2) to S_1, (x_2, x_3) to S_2, and (x_3, x_1) to S_3. We denote a secret sharing of x by $[\![x]\!]$.

From now on, servers will compute on the secret shares of x rather than on x itself. The 3 servers are capable of performing operations such as the addition of a constant, summing of secret-shared values, and multiplication by a publicly known constant by doing local computations on their respective shares. To multiply secret shared values $[\![x]\!]$ and $[\![y]\!]$, we can use the fact that

$x \cdot y = (x_1 + x_2 + x_3)(y_1 + y_2 + y_3) \mod q$. This means that S_1 computes $z_1 = x_1 \cdot y_1 + x_1 \cdot y_2 + x_2 \cdot y_1 \mod q$, S_2 computes $z_2 = x_2 \cdot y_2 + x_2 \cdot y_3 + x_3 \cdot y_2 \mod q$, and S_3 computes $z_3 = x_3 \cdot y_3 + x_3 \cdot y_1 + x_1 \cdot y_3 \mod q$. The next step is for the servers to obtain an additive secret sharing of 0 by choosing random values u_1, u_2, u_3 such that $u_1 + u_2 + u_3 = 0 \mod q$. This can be done using pseudorandom functions. Each server S_i then computes $v_i = z_i + u_i \mod q$. Finally, S_1 sends v_1 to S_3, S_2 sends v_2 to S_1, and S_3 sends v_3 to S_2. This allows the servers S_1, S_2, and S_3 to obtain the replicated secret shares (v_1, v_2), (v_2, v_3), and (v_3, v_1), respectively, of the value $v = x \cdot y$.

In Section 5, we evaluate our MPC protocol for federated SDG in the above 3PC MPC scheme, which can be proven secure against semi-honest adversaries [1]. For the malicious case, we use the MPC scheme proposed by [6] as implemented in MP-SPDZ [11].

2.2 Differential Privacy for Output Privacy

Differential privacy (DP) [7] is a privacy notion that offers theoretical bounds on how much a single individual can influence the outcome of some computation or algorithm. This means that even if an adversary has access to the output of the computation, they cannot determine whether any specific individual's data was included in the input dataset. DP bounds the difference in outputs resulting from two datasets D and D' differing in one record, set by privacy budget ϵ. (ϵ,δ)-DP is a common relaxation of the stricter ϵ-DP notion, permitting a small chance of mechanism failure quantified in δ, i.e., with probability at most δ, the mechanism may violate ϵ-differential privacy. Formally, a randomized algorithm M satisfies (ϵ,δ)-DP if and only if for any pair of neighboring datasets D and D', i.e., datasets that differ in only one record, and any $y \in Range(M)$:

$$Pr[M(D) = y] \leq e^{\epsilon} Pr[M(D') = y] + \delta \tag{1}$$

An (ϵ, δ)-DP algorithm M is usually created out of an algorithm M^* by adding noise that is proportional to the *sensitivity* of M^*, in which the sensitivity reflects the maximum impact changing a single record in D can have on the output of M^*.

2.3 Private-PGM for Synthetic Data Generation

In this paper, we adapt the marginals-based SDG algorithm Private-PGM [16] to federated settings. Private-PGM fits an undirected graphical model to differentially private noisy measurements of low-dimensional marginals, from which synthetic records are drawn by sampling the learned model.

The method operates on a dataset D with discrete features $x = \{x_1, x_2, \ldots, x_d\}$. Each feature x_i has a discrete domain Ω_i. For a subset q of features, the marginal $\mu_q(D)$ counts, for each combination of values $t \in \Omega_q$, how many times that combination appears in D, where $\Omega_q = \prod_{x_i \in q} \Omega_i$.

Private-PGM begins by computing DP marginals $\mu_q(D) + \mathcal{N}(0, \sigma_q I)$, where $\mathcal{N}$ is Gaussian noise with scale σ_q determined based on specified DP parameters (ϵ, δ). Then it estimates the joint marginal distribution that best explains all the noisy measurements. Specifically, following the implementation of Chen et al. [4] for RNA-seq data, the algorithm first computes 1-way marginals μ_g capturing the count distribution of each gene g and the label μ_y, as well as 2-way marginals $\mu_{g,y}$ capturing the joint distributions between each gene g and the label y. DP noise is then injected into these statistics based on the DP privacy budget. It then estimates the joint marginal distribution that best explains all the noisy measurements, while in parallel inferring the parameters of the graphical model using through belief propagation on a junction tree. Finally, it reconstructs a synthetic dataset from these noisy measurements using a graphical-model-based inference engine. We refer to McKenna et al. [16] for more details.

3 Related Work

Synthetic RNA-seq Data Generation. SDG algorithms have previously been developed and applied for generation of synthetic RNA-seq data in the centralized setting, i.e., when all real data used to train the generator resides with one data holder. Existing research encompasses both generation

of synthetic bulk RNA-seq data [22] and synthetic single cell RNA-seq data [14, 26]. Regarding bulk RNA-seq data, which is the focus of this paper, Chen et al. [4] produced a comprehensive comparison of SDGs, demonstrating that the marginal-based Private-PGM algorithm [16] can generate bulk RNA-seq counts with high downstream utility while offering formal differential privacy guarantees. We confirmed the effectiveness of Private-PGM through our participation in the Critical Assessment of Massive Data Analysis (CAMDA) Health Privacy Challenge [18].[1] At the conclusion of this international benchmarking competition in 2025, Private-PGM achieved the highest utility synthetic RNA-seq data (generated based on real TCGA data) of all submissions. While there may still be room for future improvement [22], these results suggest that Private-PGM is a strong starting point for developing MPC protocols to enable SDG in federated settings where data is distributed across multiple holders.

Federated Synthetic Data Generation. Research on secure SDG from distributed data sources, or federated SDG, is nascent. Most existing work focuses on image data and some on tabular data [12]. Initial federated synthesis approaches adhere closely to the original federated learning (FL) paradigm in which each client performs local neural network training and exchanges only parameter values instead of raw data with a central aggregator [29]. These approaches pay for privacy with a loss in utility compared to the centralized setting, and they are designed and evaluated for clients who each have thousands of instances available for local model training. This is in stark contrast with the scenarios that could actually benefit the most from federated SDG, provided the proper technical solutions are in place, such as the rare cancer setting in which a data holder may have data for only a handful of individuals.

Very little research has been done on training synthetic data generators across encrypted data silos. Pentyala et al. proposed the `CaPS` framework (Collaborative and Private Synthetic Data Generation) with the first MPC protocols for marginal-based tabular data generation [23]. Their method has since been shown to yield synthetic data with a substantially higher fidelity than a more traditional FL approach [15]. Experiments in [15, 23] are however performed in a so-called horizontal patitioning set-up in which it is assumed that each data holder (hospital) computes marginals for their own data on-site and subsequently sends eencrypted shares of those marginals to MPC servers to start further processing within MPC. For generation of synthetic RNA-seq data with Private-PGM, however, we need the data holders to send encrypted shares of the gene expression levels (not the marginals) to the MPC servers so that the servers can execute an MPC protocol for quantile binning across all data. Computation of marginals is subsequently performed on secret sharings of the binned data, without decrypting it. RNA-seq data is highly dimensional, resulting in a large number of marginals that needs to be computed in this way. An initial attempt to use `CaPS` for generation of synthetic RNA-seq data [24] does not scale because of an inefficient implementation of the computation of the marginals over gene expression levels. Fu et al. [9] recently identified the inefficiencies of `CaPS` and proposed to address them by optimizing secure sort-and-count operations [9]. In this paper, we present an alternative solution that lends itself well to RNA-seq data, namely replacing iterative "per sample per gene" computations by one dot product over a vector of all samples per gene. This leads to an efficient and effective solution, which to the best of our knowledge, is the first one in the literature to enable synthetic RNA-seq data generation with input privacy *and* output privacy.

4 MPC Protocol for Synthetic RNA-seq Data Generation

Formal Problem description. We consider a federated scenario where each of M data holders holds a local private dataset D_i. Each sample (i.e., each row) in D_i consists of d gene expression values as well as a label that denotes e.g. the cancer type. The data holders' goal is to leverage $\{D_1, D_2, \ldots, D_M\}$ to generate synthetic data $\widehat{D}$. This process must preserve both *input privacy*, i.e. no data holder reveals their raw records in D_i to anyone, not even to a central server, and *output privacy*, i.e., the generated $\widehat{D}$ does not leak sensitive information about the original samples in $\{D_1, D_2, \ldots, D_M\}$.

[1] `https://bipress.boku.ac.at/camda2025/competitions/camda-2025/`

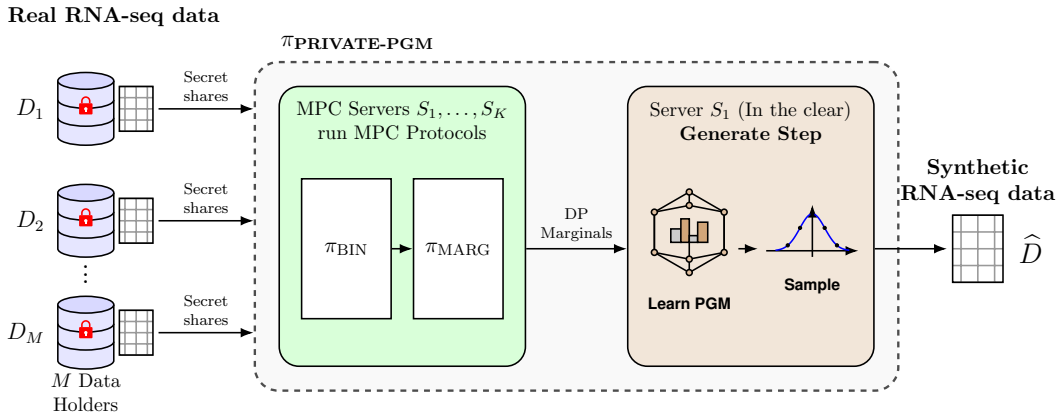


Figure 1: Overview of the proposed $\pi_{\text{PRIVATE-PGM}}$ solution for federated synthetic RNA-seq data generation. M data holders each have a dataset $(D_1, D_2, \ldots, D_M)$ with RNA-seq data from patients. Each data holder encrypts its RNA-seq data and sends secret shares to a set of MPC servers $S_1, \ldots, S_K$. The MPC servers run MPC protocols π_{BIN} and π_{MARG} to obtain encrypted marginal distributions estimated from the data *while it stays encrypted*. They subsequently run protocol $\pi_{\text{BATCH-GAUSS}}$ to perturb the encrypted estimated marginals with noise to provide differential privacy guarantees. Next the noisy marginals are decrypted by server S_1 and used to learn a probabilistic graphical model (PGM) that is subsequently sampled to generate a synthetic RNA-seq dataset $\widehat{D}$.

Protocol overview. As illustrated in Fig. 1, our proposed solution begins with each of the M data holders creating secret shares of their respective dataset D_i $(i = 1, \ldots, M)$ and sending the secret shares to K computing servers. The servers proceed with joint execution of MPC protocol $\pi_{\text{PRIVATE-PGM}}$, following the pseudocode in Protocol 1. We build upon the MPC primitives available in the MP-SPDZ library [11]. For more details, see Appendix A.

In Phase 1, the MPC servers concatenate the secret shares $[\![D_i]\!]$ from the M data holders and transpose the resulting combined matrix (Lines 1–2). This enables column-wise data access. This replaces thousands of sequential network communication rounds with batched, Single Instruction, Multiple Data (SIMD) array operations, preventing extensive instruction-unrolling. Then, the servers separate the label column from the features. The first d rows representing the gene expressions are assigned to the feature matrix $[\![D^T]\!]$ (Line 3), while the final row is split into a separate label array $[\![Y]\!]$ (Line 4). Isolating the labels from the gene expression features allows to vectorize the future computation of 2-way marginals between the two.

In Phase 2, following the preprocessing strategy in [4], on Line 5 the MPC servers execute the sub-protocol π_{BIN} for quantile binning to discretize the gene expression data to make it compatible with Private-PGM. Next, the MPC servers execute the sub-protocol π_{MARG} for computing 1-way and 2-way marginals over the encrypted discretized data (Line 6), and $\pi_{\text{BATCH-GAUSS}}$ for perturbing the computed marginals with noise to provide differential privacy (DP) guarantees (Line 7). We describe each of these sub-protocols in more detail below.

The noisy marginals computed in Phase 2 satisfy DP and can therefore be revealed (analogous to decryption) without compromising individual privacy. This follows from the *post-processing property* of DP which states that any computation applied to the output of a DP method preserves the same privacy guarantees and cannot increase privacy leakage [7]. In line with this, in Phase 3, all MPC servers reveal their secret shares of the noisy marginals (Line 8) and bin means (Line 9) by sending them to a designated data release server, S_1. Server S_1 combines the shares to obtain plaintext privatized marginals and bin means. In Phase 4, these are used to locally train a probabilistic graphical model to sample and generate the synthetic data $\widehat{D}$ (Lines 10-11). On Line 10, S_1 trains a PGM on the discrete data to generate discrete synthetic data [16]. On line Line 11, S_1 de-bins the discrete synthetic data using the released DP means to map it to its original continuous space.

Secure quantile binning π_{BIN}. The Private-PGM algorithm for SDG assumes that the features in the training data are categorical [16]. To process RNA-seq data with Private-PGM, Chen et

Protocol 1 $\pi_{\text{PRIVATE-PGM}}$: Main MPC Protocol for Synthetic Data Generation

Require: Secret shares of data $[\![D_i]\!]$, number N_i of samples in D_i, number of data holders M, number of classes C, number of genes d, scale σ

Ensure: Synthetic dataset $\widehat{D}$

▷ **Phase 1: Data Preparation**

1: $N \leftarrow \sum_{i=1}^{M} N_i$ ▷ Total number of samples

2: $[\![D^T_{\text{combined}}]\!] \leftarrow \text{Transpose}\left(\text{Concat}_{i=1}^{M}([\![D_i]\!])\right)$

3: $[\![D^T]\!] \leftarrow [\![D^T_{\text{combined}}[0 \ldots d-1]]\!]$ ▷ Gene features

4: $[\![Y]\!] \leftarrow [\![D^T_{\text{combined}}[d]]\!]$ ▷ Labels

▷ **Phase 2: Secure Quantile Binning and Computation of Noisy Marginals**

5: $[\![D^T]\!], [\![\mu_{\text{mean}}]\!] \leftarrow \pi_{\text{BIN}}([\![D^T]\!], N, d)$ ▷ See Protocol 2

6: $[\![\mu_g]\!], [\![\mu_y]\!], [\![\mu_{g,y}]\!] \leftarrow \pi_{\text{MARG}}([\![D^T]\!], [\![Y]\!], N, C, d)$ ▷ See Protocol 3

7: $[\![\vec{\mu}_{\text{noisy}}]\!] \leftarrow \pi_{\text{BATCH-GAUSS}}([\![\mu_g]\!], [\![\mu_y]\!], [\![\mu_{g,y}]\!], \sigma)$ ▷ See Protocol 4

▷ **Phase 3: Release of Differentially Private Marginals**

8: $\tilde{\mu}_g, \tilde{\mu}_y, \tilde{\mu}_{g,y} \leftarrow \pi_{\text{REVEAL}}([\![\vec{\mu}_{\text{noisy}}]\!])$ ▷ Revealed to S_1

9: $\tilde{\mu}_{\text{mean}} \leftarrow \pi_{\text{REVEAL}}([\![\mu_{\text{mean}}]\!])$ ▷ Revealed to S_1

▷ **Phase 4: Synthetic Data Generation**

10: $\widehat{D}_{\text{discrete}} \leftarrow \text{Private-PGM}(\tilde{\mu}_g, \tilde{\mu}_y, \tilde{\mu}_{g,y})$ ▷ Generate discrete data locally by S_1

11: $\widehat{D} \leftarrow \text{Inverse-Binning}(\widehat{D}_{\text{discrete}}, \tilde{\mu}_{\text{mean}})$ ▷ De-bin using DP means

12: **return** $\widehat{D}$

al. suggested discretization of gene expression counts into 4 quantiles [4]. To perform such discretization efficiently over encrypted values in MPC, we adapt the binning logic from Pentyala et al. [24]. The protocol π_{BIN} takes the gene expression matrix $[\![D^T]\!]$, maps each gene into 4 quantile bins, and computes the mean value of each bin across all samples (Lines 2–15 in Protocol 2). For each gene g, the servers first extract the expression values into an array $[\![V_g]\!]$ (Line 3) and securely sort them (Line 4) to extract the quartile boundaries $[\![Q_0]\!], [\![Q_1]\!], [\![Q_2]\!]$ (Line 5). They then perform secure comparisons (π_{LT}) against these boundaries to create indicator arrays (Line 6). By combining these, a bin assignment array $[\![\vec{B}_g]\!]$ is computed, mapping every sample's value for gene g to a discrete bin representing which quantile it falls into $\{0, 1, 2, 3\}$ (Line 7). For example, if a sample's value falls below the first quartile $[\![Q_0]\!]$, all three indicators yield 1, mapping it to bin $3 - 1 - 1 - 1 = 0$. $[\![\vec{B}_g]\!]$ holds the secret shares of the newly binned features. Next, for each bin b (Line 8), a Boolean mask $[\![\vec{m}_b]\!]$ is generated, representing whether the sample falls into bin b (Line 9). Using these masks, the protocol securely counts the frequencies for each bin (Line 10). It then computes the mean expression of gene g for samples in bin b by multiplying the binary bin mask $[\![\vec{m}_b]\!]$ by the continuous expression values $[\![V_g]\!]$ to securely sum only the values of the samples belonging to bin b (Line 11). This filtered sum is then divided by the total number of samples in that bin, $[\![\text{count}_b]\!]$, to yield the exact bin mean (Line 12). Before returning, the original gene expression values are replaced with their discrete bin indices (Line 14).

We optimize this process for MPC to minimize network latency and multiplicative depths, which otherwise slow down compilation and execution times. Unlike previous sequential approaches that process one sample at a time [24], we replace the iterative loops over each sample previously used to compute bin frequencies and means with vector dot products (Protocol 2, Lines 8–13). This collapses the original per-sample, per-gene iterative computations to just one dot product over a vector of N samples per gene.

Secure computation of marginals π_{MARG}. The servers execute MPC protocol π_{MARG} to compute the 1-way marginals (gene frequencies) and 2-way marginals (gene-label correlations). To achieve this efficiently on secret-shared data, we use indicator polynomials to convert a discrete bin index from $\{0, 1, 2, 3\}$ computed by π_{BIN} into a one-hot encoded vector. This is necessary because, in MPC, servers cannot directly use secret-shared bin indices to access arrays without leaking the underlying data. By evaluating the indicator polynomials described below, the servers can construct the indicator vectors entirely through additions and multiplications, avoiding computationally expensive equality check operations, which significantly improves the efficiency of π_{MARG}. For a

Protocol 2 π_{BIN}: MPC Protocol for Quantile Binning

Require: Transposed secret-shared dataset $[\![D^T]\!]$, number of samples N, number of genes d
Ensure: Secret-shared binned dataset $[\![D^T]\!]$, secret-shared bin means $[\![\mu_{\text{mean}}]\!]$

$[\![\vec{1}_N]\!] \leftarrow$ Array of 1s of size N
for every gene $g \in \{0, \ldots, d-1\}$ **do**
  $[\![V_g]\!] \leftarrow [\![D^T[g]]\!]$
  $[\![S_g]\!] \leftarrow \pi_{\text{SORT}}([\![V_g]\!])$
  Extract quartiles: $[\![Q_0]\!], [\![Q_1]\!], [\![Q_2]\!] \leftarrow [\![S_g[\lfloor 0.25N \rfloor]]\!], [\![S_g[\lfloor 0.50N \rfloor]]\!], [\![S_g[\lfloor 0.75N \rfloor]]\!]$
  $[\![\vec{c}_0]\!] \leftarrow \pi_{\text{LT}}([\![V_g]\!], [\![Q_0]\!])$; $[\![\vec{c}_1]\!] \leftarrow \pi_{\text{LT}}([\![V_g]\!], [\![Q_1]\!])$; $[\![\vec{c}_2]\!] \leftarrow \pi_{\text{LT}}([\![V_g]\!], [\![Q_2]\!])$
  $[\![\vec{B}_g]\!] \leftarrow \vec{3} - [\![\vec{c}_2]\!] - [\![\vec{c}_1]\!] - [\![\vec{c}_0]\!]$
  for $b \in \{0, 1, 2, 3\}$ **do**
    $[\![\vec{m}_b]\!] \leftarrow \pi_{\text{EQ}}([\![\vec{B}_g]\!], b)$
    $[\![\text{count}_b]\!] \leftarrow [\![\vec{m}_b]\!] \cdot \vec{1}_N$
    $[\![T]\!] \leftarrow \pi_{\text{DOT}}([\![\vec{m}_b]\!], [\![V_g]\!])$
    $[\![\mu_{\text{mean}}[g][b]]\!] \leftarrow \pi_{\text{DIV}}([\![T]\!], [\![\text{count}_b]\!])$
  end for
  $[\![D^T[g]]\!] \leftarrow [\![\vec{B}_g]\!]$
end for
return $[\![D^T]\!], [\![\mu_{\text{mean}}]\!]$

target bin $f \in \{0, 1, 2, 3\}$, the polynomial $G_f(x)$ is defined as:

$$G_0(x) = \frac{(1-x)(2-x)(3-x)}{6}; \quad G_1(x) = \frac{x(2-x)(3-x)}{2}; \quad G_2(x) = \frac{x(x-1)(3-x)}{2}; \quad G_3(x) = \frac{x(x-1)(x-2)}{6}$$

By evaluating these polynomials, $G_f(x) = 1$ if $x = f$, and 0 otherwise.

This allows us to express the 2-way marginal count $\mu_{g,y}(f, c)$ for a gene g at bin f and class label c as a sum of products. If $L_c(y_i)$ represents a Boolean indicator for whether the i-th sample belongs to class c, the 2-way marginal is computed as $\mu_{g,y}(f, c) = \sum_{i=1}^{N} G_f(x_i) \cdot L_c(y_i)$.

Protocol 3 π_{MARG}: MPC Protocol for Computing Marginals

Require: Secret-shared binned dataset $[\![D^T]\!]$, target labels $[\![Y]\!]$, number of samples N, number of classes C, number of genes d
Ensure: Exact 1-way marginals $[\![\mu_g]\!], [\![\mu_y]\!]$, and 2-way marginals $[\![\mu_{g,y}]\!]$

1: $[\![\vec{1}_N]\!] \leftarrow$ Array of 1s of size N
2: **for** $c \in \{0, \ldots, C-1\}$ **do**
3: $[\![\vec{L}_c]\!] \leftarrow \pi_{\text{EQ}}([\![Y]\!], c)$
4: $[\![\mu_y[c]]\!] \leftarrow [\![\vec{L}_c]\!] \cdot \vec{1}_N$
5: **end for**
6: **for** every gene $g \in \{0, \ldots, d-1\}$ **do**
7: $[\![\vec{x}]\!] \leftarrow [\![D^T[g]]\!]$
8: $[\![\vec{s}_1]\!] \leftarrow \vec{1} - [\![\vec{x}]\!]$; $[\![\vec{s}_{11}]\!] \leftarrow [\![\vec{x}]\!] - \vec{1}$; $[\![\vec{s}_2]\!] \leftarrow \vec{2} - [\![\vec{x}]\!]$; $[\![\vec{s}_{21}]\!] \leftarrow [\![\vec{x}]\!] - \vec{2}$; $[\![\vec{s}_3]\!] \leftarrow \vec{3} - [\![\vec{x}]\!]$
9: $[\![\vec{G}_0]\!] \leftarrow \pi_{\text{MUL}}([\![\vec{s}_1]\!], [\![\vec{s}_2]\!], [\![\vec{s}_3]\!])/6;$
10: $[\![\vec{G}_1]\!] \leftarrow \pi_{\text{MUL}}([\![\vec{x}]\!], [\![\vec{s}_2]\!], [\![\vec{s}_3]\!])/2;$
11: $[\![\vec{G}_2]\!] \leftarrow \pi_{\text{MUL}}([\![\vec{x}]\!], [\![\vec{s}_{11}]\!], [\![\vec{s}_3]\!])/2;$
12: $[\![\vec{G}_3]\!] \leftarrow \pi_{\text{MUL}}([\![\vec{x}]\!], [\![\vec{s}_{11}]\!], [\![\vec{s}_{21}]\!])/6$
13: **for** $f \in \{0, 1, 2, 3\}$ **do**
14: $[\![\mu_g[g][f]]\!] \leftarrow [\![\vec{G}_f]\!] \cdot \vec{1}_N$ ▷ Local dot product
15: **for** $c \in \{0, \ldots, C-1\}$ **do**
16: $[\![\mu_{g,y}[g][f \cdot C + c]]\!] \leftarrow \pi_{\text{DOT}}([\![\vec{G}_f]\!], [\![\vec{L}_c]\!])$
17: **end for**
18: **end for**
19: **end for**
20: **return** $[\![\mu_g]\!], [\![\mu_y]\!], [\![\mu_{g,y}]\!]$

First, the servers iterate over the C target classes to generate Boolean indicator vectors $[\![\vec{L}_c]\!]$ for each class (Line 3 in Protocol 3), summing them via dot product to calculate the exact 1-way marginals for the label array $[\![\mu_y]\!]$ (Line 4). Next, for each gene g (Line 6), the servers extract the corresponding binned array $[\![\vec{x}]\!]$ (Line 7), calculate the intermediate values (Line 8), and compute the polynomials to generate the one-hot encoded arrays $[\![\vec{G}_0]\!]$ to $[\![\vec{G}_3]\!]$ for the four bins in $\{0, 1, 2, 3\}$

(Line 9). Finally, for each target bin $f \in \{0, 1, 2, 3\}$, the servers compute 1-way marginals of the gene by taking the dot product between the encoded array $[\![\vec{G}_f]\!]$ and the vector of ones (Line 11). To compute the 2-way marginals between the gene and the target labels, the servers evaluate our previously defined formula by computing dot products between $[\![\vec{G}_f]\!]$ and the label indicator vectors $[\![\vec{L}_c]\!]$ for each class c (Lines 12–13).

This approach improves the compilation and execution speed of the protocol compared to an earlier attempt [24], which sequentially iterated over individual samples to compute the marginals and inject differential privacy (DP) noise element-wise. Such sequential processing resulted in a high number of repeated operations and network communication rounds, leading to slower execution speeds and large memory overhead. To resolve this, we apply a dot-product optimization: instead of sequentially iterating over individual samples, we compute the 1-way and 2-way marginal counts securely with vectorized dot products between the feature and the label indicator vectors (Line 11 for 1-way marginals and Line 13 for 2-way marginals).

Secure perturbation with noise for differential privacy with $\pi_{\text{BATCH-GAUSS}}$. We extend this vectorization to the DP noise phase as well. After the computing servers generate secret-shared Irwin-Hall approximated Gaussian noise (see Protocol 5 based on [23]), we flatten the secret-shared marginal vectors and inject this noise in a single vectorized addition in Protocol 4. This replaces thousands of element-wise network requests with one collective batched operation, resulting in significantly faster compilation.

Protocol 4 $\pi_{\text{BATCH-GAUSS}}$: MPC Batched Gaussian Addition

Require: Exact marginals $[\![\mu_g]\!], [\![\mu_y]\!], [\![\mu_{g,y}]\!]$, noise scale σ
Ensure: Flat array of noisy marginals $[\![\vec{\mu}_{\text{noisy}}]\!]$
1: $[\![\vec{\mu}_{\text{flat}}]\!] \leftarrow \text{Flatten}([\![\mu_g]\!], [\![\mu_y]\!], [\![\mu_{g,y}]\!])$
2: $M \leftarrow \text{length}([\![\vec{\mu}_{\text{flat}}]\!])$
3: $[\![\vec{\mathcal{N}}]\!] \leftarrow \pi_{\text{GAUSS}}(M)$ ▷ For the MPC-friendly protocol, see Appendix B
4: $[\![\vec{\mu}_{\text{noisy}}]\!] \leftarrow [\![\vec{\mu}_{\text{flat}}]\!] + \sigma \cdot [\![\vec{\mathcal{N}}]\!]$
5: **return** $[\![\vec{\mu}_{\text{noisy}}]\!]$

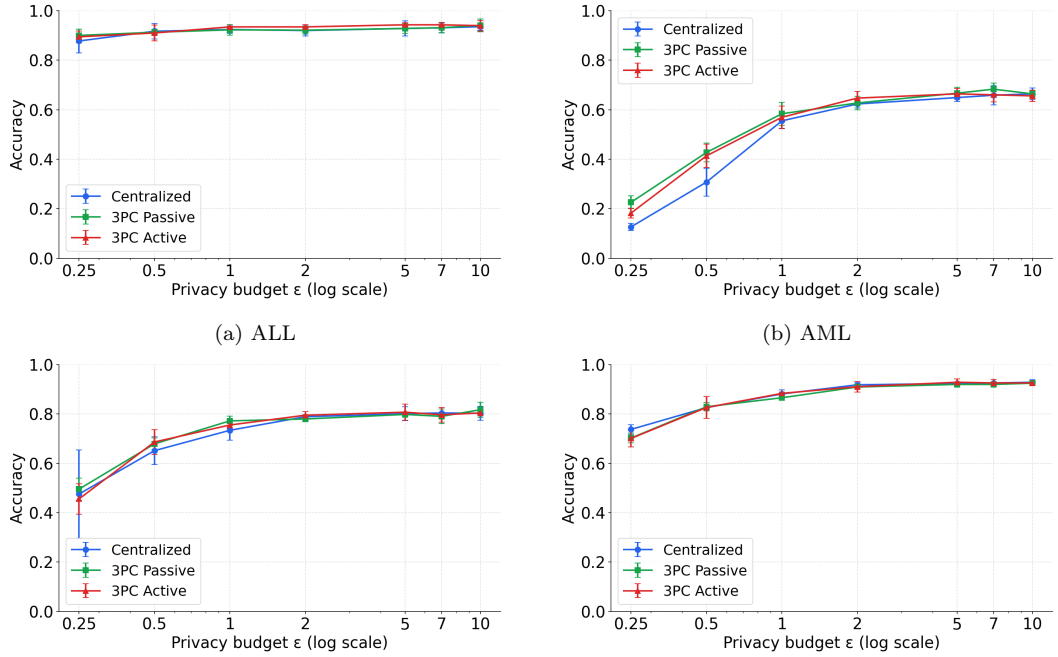
5 Empirical Evaluation Results

Datasets. We evaluate our approach on four datasets across different cancer types. The first dataset, which we designate as ALL, contains 1,181 samples across four leukemia subtypes [27]; it was used in a previous study of centralised SDG methods for genomic data [4]. The second, which we designate as AML, is a more recent dataset of Acute Myeloid Leukemia (AML) subtypes containing 1,224 bulk RNA-seq samples across 14 subtype labels [25]. We also evaluate our method on two public benchmark datasets from the CAMDA 2025 competition on privacy-preserving genomic data generation: TCGA-BRCA, a breast cancer dataset with 1,089 samples across five subtypes, and TCGA-COMBINED, a larger collection of 4,323 samples spanning ten cancer tissue types. Together, these datasets span diverse domains, class distributions, and label cardinalities, enabling a thorough evaluation of our method under conditions analogous to real-world federated rare disease settings.

We follow the preprocessing pipeline of Chen et al. (2024) [4]. We apply DESeq2 normalization [13] to minimize the influence of varying sequencing depths and RNA compositions, followed by gene filtering and ensuring log-normalization to account for the well-known skewness of RNA-seq counts. The categorical labels are encoded as integers. After filtering, the datasets ALL, AML, TCGA-BRCA, and TCGA-COMB were reduced to 958, 1,000, 978, and 978 features, respectively.

Experimental Setup. To evaluate the fidelity and utility of the synthetic data, we divide each dataset into a training set D_{train} and a held-out test set D_{test} using 80/20 data split. For our experiments across the different numbers of features, such as one in Figure 3, we randomly subsample the genes to some number X. Next we distribute the samples in D_{train} among M data holders. Because all operations in $\pi_{\text{PRIVATE-PGM}}$ are performed by the MPC servers over an encrypted version

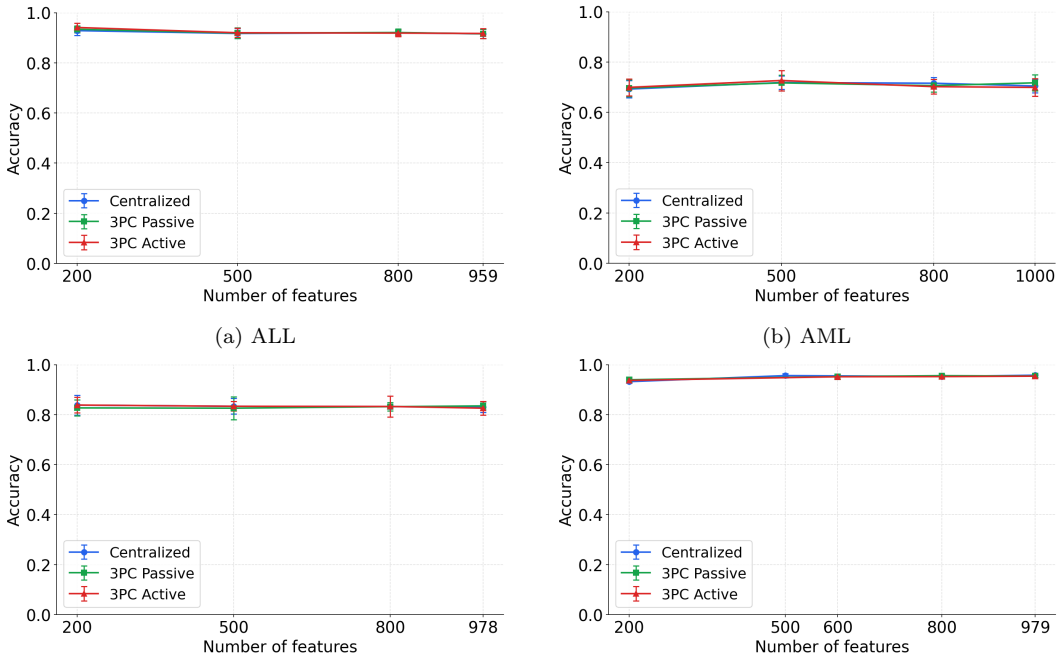
of the entire dataset D_{train}, the results in this section are agnostic to the value of M. We implement $\pi_{\text{PRIVATE-PGM}}$ in MP-SPDZ using both passive and active 3PC schemes, i.e. for $K = 3$ MPC servers[2].



(a) ALL (b) AML

(c) TCGA-BRCA (d) TCGA-COMBINED

Figure 2: Classification accuracy across different privacy budgets (ϵ). Larger values of ϵ provide less privacy.



(a) ALL (b) AML

(c) TCGA-BRCA (d) TCGA-COMBINED

Figure 3: Classification accuracy across different numbers of genes, comparing 3PC Active and 3PC Passive schemes against centralized Private-PGM.

[2] `https://github.com/Filienko/Pro-GENE-GEN-MPC`

Evaluation Metrics. Following prior work [4, 18], we assess synthetic data quality across four dimensions. *Utility* is evaluated via a Train-on-Synthetic, Test-on-Real (TSTR) framework with classification accuracy using logistic regression. *Fidelity* is quantified by Wasserstein distance between the gene counts in synthetic and real data. *Biological value* is assessed by differential expression preservation (TPR of real differential expressed genes recovered in synthetic data). *Privacy*, following CAMDA, is assessed by Distance to Closest Record (DCR), which is the mean Euclidean distance from each synthetic sample to its nearest real neighbour. Closer DCR reflects more realistic looking and hence potentially sensitive data.

Table 1: MPC protocol's compile and execution time (seconds) by dataset and number of features, averaged over 3 runs

		3PC Passive				**3PC Active**			
Dataset	**Time**	**200**	**500–600**	**800**	**959–1000**	**200**	**500–600**	**800**	**959–1000**
BRCA (N=1,089)	Compile	2.14	2.22	2.99	2.59	2.06	3.62	2.46	2.76
	Execute	23.42	55.94	88.04	117.43	983.28	2735.95	4465.14	4800.20
ALL (N=1,181)	Compile	2.13	2.41	2.63	2.83	2.22	3.36	3.14	2.89
	Execute	25.20	57.76	90.92	117.30	1016.58	2915.58	4698.88	5627.41
AML (N=1,224)	Compile	3.31	3.84	4.51	4.64	3.28	3.84	6.28	4.68
	Execute	25.67	61.92	100.09	133.49	1077.39	2627.35	7013.80	5641.11
COMB (N=4,323)	Compile	7.49	8.20	8.27	8.64	7.64	8.03	8.35	8.99
	Execute	118.10	249.35	324.88	426.92	4337.63	12758.33	16330.31	20792.39

Feature counts 200/500/800/959–1000 for ALL, BRCA, AML; 200/600/800/979 for COMB. 3PC Passive = semi-honest (ring); 3PC Active = malicious (mal-rep-ring). N is number of samples in the dataset.

Discussion. Our evaluation demonstrates that the proposed MPC protocol (input privacy and output privacy) achieves performance on par with both non-private (no input privacy, no output privacy) and centralized (only output privacy) baselines. Specifically, as the privacy budget (ϵ) increases, the synthetic data approaches the utility and biological value of non-private real data, in the downstream classification task accuracy (Fig. 2), differential expression preservation (Fig. 5 in Appendix C), and one dimensional marginal distribution similarity (Fig. 4 in Appendix C). Our approach consistency delivers fidelity and utility comparable to the centralized setting, as can be seen in Fig. 3 and Fig. 4 in Appendix C. Importantly, these privacy-preserving benefits are achieved at a low computational cost; as detailed in Tab. 1, protocol compilation times are negligible (under 10 seconds), and execution times remain feasible across both active and passive attacker scenarios, even as the number of features increase [3].

6 Conclusion

We introduced an efficient MPC-based protocol for privacy-preserving generation of synthetic bulk RNA-seq data in federated settings, combining strong input privacy with formal differential privacy guarantees for released data. By re-engineering Private-PGM for encrypted, high-dimensional genomics data and leveraging vectorized MPC primitives, our approach attains utility and biological fidelity comparable to centralized baselines while remaining computationally practical under both passive and active adversary models. Empirical results across multiple real-world cancer datasets demonstrate that high-quality synthetic genomic data can be generated jointly across institutions without disclosing raw patient records. We view this work as a concrete step toward alleviating persistent data-sharing barriers in genomics and enabling secure, scalable collaboration in rare disease and multi-site biomedical research.

[3] We ran our experiments on a machine with 128 GB RAM, 2 NVIDIA RTX A6000 GPUs, and AMD Ryzen Threadripper PRO 5965WX 24-Cores

Acknowledgements This research was supported by NSF #2451163, NIH 1OT2OD032581, NSF NAIRR 240485 (Cloudbank AWS), NSF NAIRR 240091 (TACC Frontera), the UW Tacoma Founders Endowment, and the UW eScience Institute.

References

[1] T. Araki, J. Furukawa, Y. Lindell, A. Nof, and K. Ohara. High-throughput semi-honest secure three-party computation with an honest majority. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 805–817, 2016.

[2] R. Canetti. Security and Composition of Multiparty Cryptographic Protocols. *Journal of Cryptology*, 13 (1):143–202, 2000.

[3] O. Catrina and A. Saxena. Secure computation with fixed-point numbers. In *14th International Conference on Financial Cryptography and Data Security*, volume 6052 of *Lecture Notes in Computer Science*, pages 35–50. Springer, 2010.

[4] D. Chen, M. Oestreich, T. Afonja, R. Kerkouche, M. Becker, and M. Fritz. Towards biologically plausible and private gene expression data generation. In *Proceedings in Privacy-Enhancing Technologies*, volume 2, pages 531–554, 2024.

[5] R. Cramer, I. Damgård, and J. B. Nielsen. *Secure Multiparty Computation and Secret Sharing*. Cambridge University Press, 2015.

[6] A. Dalskov, D. Escudero, and M. Keller. Fantastic four: Honest-majority four-party secure computation with malicious security. In *USENIX 2021*, pages 2183–2200, 2021.

[7] C. Dwork and A. Roth. The algorithmic foundations of differential privacy. *Foundations and Trends® in Theoretical Computer Science*, 9(3–4):211–407, 2014.

[8] H. Eerikson, M. Keller, C. Orlandi, P. Pullonen, J. Puura, and M. Simkin. Use Your Brain! Arithmetic 3PC for Any Modulus with Active Security. In *1st Conference on Information-Theoretic Cryptography*, 2020.

[9] Y. Fu, T. Gu, E. Shi, and T. Wang. Towards Vertically Distributed Differentially Private Synthetic Data Generation. In *Theory and Practice of Differential Privacy Workshop Series*, 2025.

[10] Y. Hu, F. Wu, Q. Li, Y. Long, G. Garrido, C. Ge, B. Ding, D. Forsyth, B. Li, and D. Song. SoK: Privacy-preserving data synthesis. In *IEEE Symposium on Security and Privacy (SP)*, pages 4696–4713, 2024.

[11] M. Keller. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1575–1590, 2020.

[12] C. Little, M. Elliot, and R. Allmendinger. Federated learning for generating synthetic data: a scoping review. *International Journal of Population Data Science*, 8(1):2158, 2023.

[13] M. I. Love, W. Huber, and S. Anders. Moderated estimation of fold change and dispersion for RNA-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014.

[14] E. Luo, M. Hao, L. Wei, and X. Zhang. scDiffusion: conditional generation of high-quality single-cell data using diffusion model. *Bioinformatics*, 40(9):btae518, 2024.

[15] S. Maddock, G. Cormode, and C. Maple. FLAIM: AIM-based synthetic data generation in the federated setting. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2165–2176, 2024.

[16] R. McKenna, D. Sheldon, and G. Miklau. Graphical-model based estimation and inference for differential privacy. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pages 4435–4444. PMLR, 2019.

[17] R. McKenna, B. Mullins, D. Sheldon, and G. Miklau. AIM: an adaptive and iterative mechanism for differentially private synthetic data. *Proceedings of the VLDB Endowment*, 15(11):2599–2612, 2022.

[18] S. Menzies, D. Filienko, S. Golob, S. Pentyala, J. Banerjee, L. Foschini, and M. de Cock. Synthetic Data Generation for bulk RNA-seq Data: A CAMDA Health Challenge Analysis. In *33rd Conference on Intelligent Systems for Molecular Biology & 24th European Conference on Computational Biology (ISMB/ECCB)*, 2025.

[19] NIH. Implementation update for data management and access practices under the genomic data sharing policy. Available at: grants.nih.gov/grants/guide/notice-files/NOT-OD-24-157.html, 2024.

[20] NIH. Request for information on responsibly developing and sharing generative artificial intelligence tools using NIH controlled access data. Available at: grants.nih.gov/grants/guide/notice-files/NOT-OD-25-118.html, May 30, 2025.

[21] M. Oestreich, D. Chen, J. L. Schultze, M. Fritz, and M. Becker. Privacy considerations for sharing genomics data. *EXCLI journal*, 20:1243, 2021. PMCID: PMC8326502.

[22] H. Öztürk, T. Afonja, J. Jälkö, R. Binkyte, P. Rodriguez-Mier, S. Lobentanzer, A. Wicks, J. Kreuer,

S. Ouaari, N. Pfeifer, et al. Towards Useful and Private Synthetic Omics: Community Benchmarking of Generative Models for Transcriptomics Data. *bioRxiv*, pages 2026–03, 2026.

[23] S. Pentyala, M. Pereira, and M. De Cock. CaPS: Collaborative and Private Synthetic Data Generation from Distributed Sources. In *International Conference on Machine Learning*, volume 235 of *PMLR*, 2024.

[24] S. Pentyala, G. Sitaraman, T. Claar, and M. De Cock. End to end collaborative synthetic data generation. *arXiv preprint arXiv:2412.03766*, 2024.

[25] J. F. Severens, E. O. Karakaslar, B. A. van der Reijden, E. Sánchez-López, R. R. van den Berg, C. J. Halkes, P. van Balen, H. Veelken, M. J. Reinders, M. Griffioen, et al. Mapping AML heterogeneity-multi-cohort transcriptomic analysis identifies novel clusters and divergent ex-vivo drug responses. *Leukemia*, 38(4):751–761, 2024.

[26] D. Song, Q. Wang, G. Yan, T. Liu, T. Sun, and J. J. Li. scDesign3 generates realistic in silico data for multimodal single-cell and spatial omics. *Nature Biotechnology*, 42(2):247–252, 2024.

[27] S. Warnat-Herresthal, K. Perrakis, B. Taschler, M. Becker, K. Baßler, M. Beyer, P. Günther, J. Schulte-Schrepping, L. Seep, K. Klee, et al. calable Prediction of Acute Myeloid Leukemia Using High-Dimensional Machine Learning and Blood Transcriptomics. *iScience*, 23(1), 2020.

[28] H. Watson, J. Gallifant, Y. Lai, A. P. Radunsky, C. Villanueva, N. Martinez, J. Gichoya, U. K. Huynh, and L. A. Celi. Delivering on NIH data sharing requirements: avoiding open data in appearance only. *BMJ Health & Care Informatics*, 30(1), 2023.

[29] B. Xin, Y. Geng, T. Hu, S. Chen, W. Yang, S. Wang, and L. Huang. Federated synthetic data generation with differential privacy. *Neurocomputing*, 468, 2022.

A MPC Primitives

In our proposed MPC protocol for federated synthetic data generation, we build upon the cryptographic primitives available in the MP-SPDZ [11] library. We use the following primitives as sub-protocols:

- Secure random number generation π_{RAND}: At the start of this protocol, the parties (MPC servers) agree on a fractional precision l for the power 2 ring representation; at the end, they have a secret sharing $[\![r]\!]$ of a uniformly distributed random number r, generated securely via the bitwise XOR of locally generated random bits.
- Secure sorting π_{SORT}: At the start of this protocol, the parties have a secret-shared array of n elements $[\![\mathbf{x}]\!]$ (and optionally an associated array of payloads $[\![\mathbf{y}]\!]$); at the end, they have a secret-shared array $[\![\mathbf{x}']\!]$ where the elements are sorted according to a specified order, without revealing the underlying values or the resulting data permutation to any party.
- Secure multiplication π_{MUL}: At the start of this protocol, the parties have secret-shared values $[\![a]\!]$ and $[\![b]\!]$; at the end, they have a secret sharing of the product $[\![c]\!]$, where $c = a \cdot b$.
- Secure division π_{DIV}: At the start of this protocol, the parties have secret-shared values $[\![a]\!]$ and $[\![b]\!]$; at the end, they have a secret sharing of the quotient $[\![c]\!]$, where $c = a/b$.
- Secure dot product π_{DOT}: At the start of this protocol, the parties have secret shares of two vectors $[\![\mathbf{a}]\!]$ and $[\![\mathbf{b}]\!]$ of length n; at the end, they have a secret sharing of their dot product $[\![c]\!]$, where $c = \sum_{i=1}^{n} a_i b_i$.
- Secure equality test π_{EQ}: At the start of this protocol, the parties have a secret-shared values $[\![a]\!]$ and $[\![b]\!]$; at the end, they have a secret shared $[\![c]\!]$, where $c = 1$ if $a = b$, and $c = 0$ otherwise.
- Secure less than test π_{LT}: At the start of this protocol, the parties have secret-shared values $[\![a]\!]$ and $[\![b]\!]$; at the end, they have a secret shared $[\![c]\!]$, where $c = 1$ if $a < b$, and $c = 0$ otherwise.
- Secure reveal π_{REVEAL}: At the start of this protocol, the parties have a secret-shared value $[\![a]\!]$, with the shares of a spread across K servers; at the end, the shares are aggregated and reconstructed so that the parties have the decrypted plaintext value a.

We build our solution using these cryptographic primitives to ensure that our final solution is also cryptographically secure, based on the universal composition theorem, which states that a protocol remains secure when composed with other or the same MPC protocols [2].

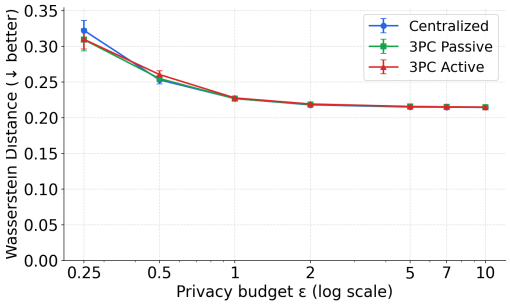
B MPC Protocol for Irwin-Hall Approximation of Gaussian Noise

Protocol 5 π_{GAUSS}: MPC Irwin-Hall Approximation of Gaussian Noise [23]

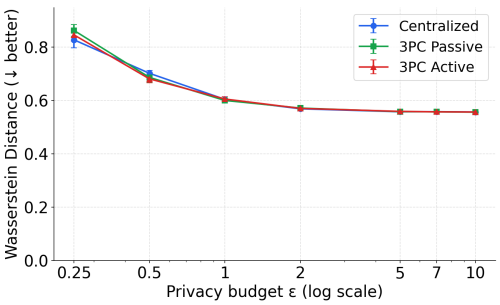
Require: Target array size Z
Ensure: Secret shared array $[\![\vec{\mathcal{N}}]\!]$ of size Z sampled from approximate $\mathcal{N}(0,1)$

```
[[N⃗]] ← 0⃗
for k ← 1 to 12 do
    [[N⃗]] ← [[N⃗]] + π_RAND(0, 1, size = Z)        ▷ Sample from uniform distribution U(0,1)
end for
[[N⃗]] ← [[N⃗]] − 6⃗
return [[N⃗]]
```

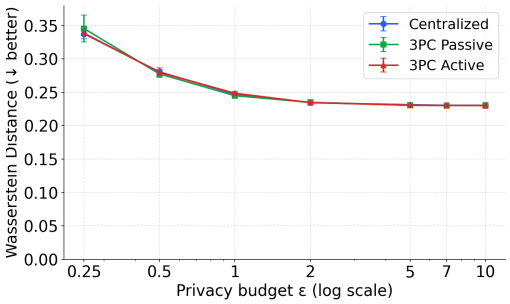
C Additional Experimental Results



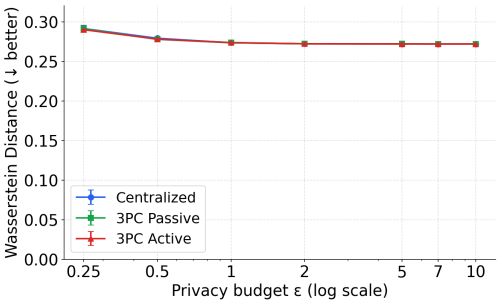
(a) ALL



(b) AML

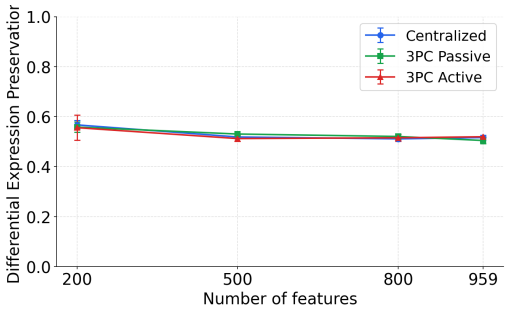


(c) TCGA-BRCA

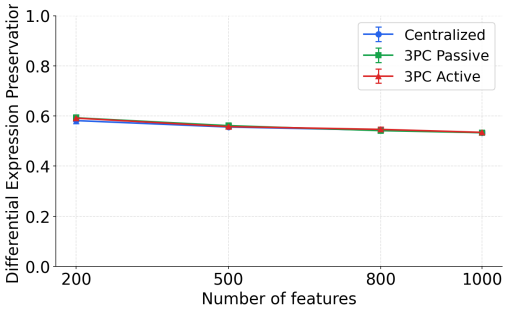


(d) TCGA-COMBINED

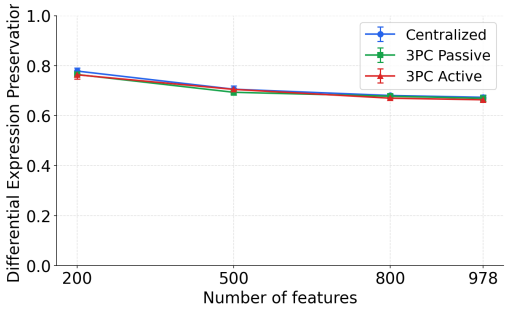
Figure 4: Wasserstein distance (lower is better) across noise budgets across all datasets, demonstrating the distance between the original and the synthetic marginals.



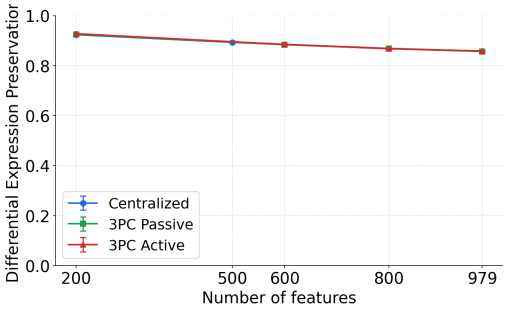
(a) ALL



(b) AML

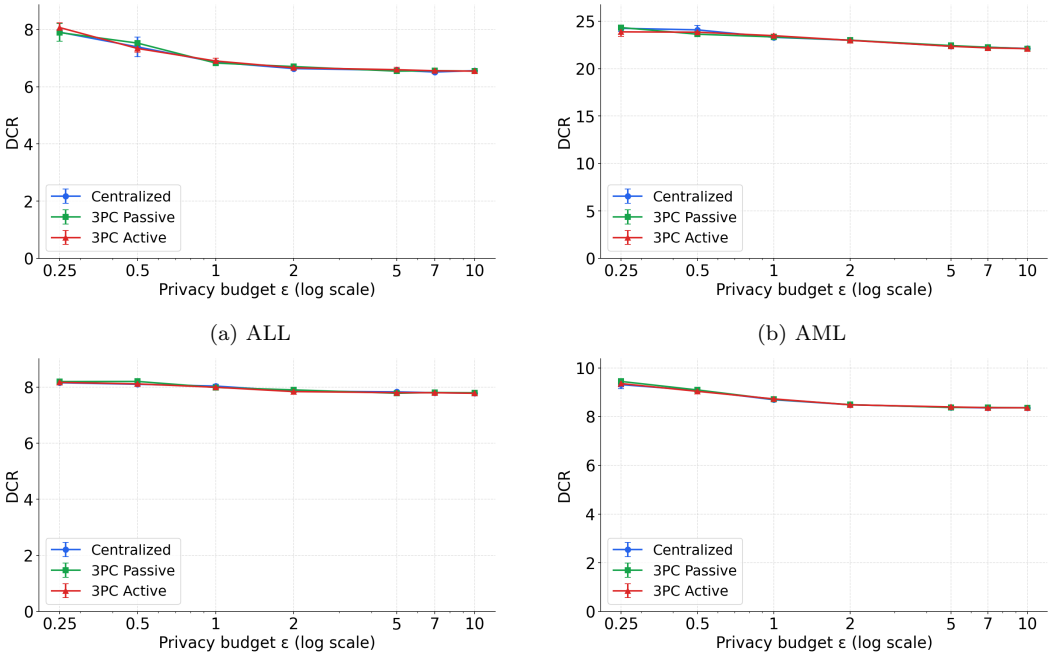


(c) TCGA-BRCA



(d) TCGA-COMBINED

Figure 5: Differentially expressed gene preservation (DETPR; higher is better) across the number of genes, demonstrating the share of the original differentially expressed genes preserved in the synthetic data.



(a) ALL

(b) AML

(c) TCGA-BRCA

(d) TCGA-COMBINED

Figure 6: Distance to Closest Record (DCR; higher is better) across different privacy budgets (ϵ), acting as a proxy for memorization where the distance of 0 would entail that the generated synthetic data is identical to the real data and hence provides no privacy protection.